

EDR (Endpoint Detection & Response) agent

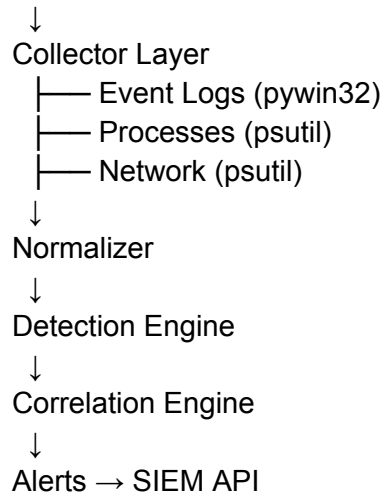
PROJECT: Windows EDR-Style Monitoring Agent

What it does

- ✓ Collects Windows logs (auth, process)
 - ✓ Monitors running processes (PID, PPID)
 - ✓ Captures network connections
 - ✓ Correlates activity
 - ✓ Generates alerts
 - ✓ Sends to SIEM (Node.js)
-

1 ARCHITECTURE

[Windows APIs + System Data]



2 PROJECT STRUCTURE

```
edr-agent/
├── main.py
├── collectors/
│   ├── windows_logs.py
│   └── process_monitor.py
```

```
|   └─ network_monitor.py
| └─ detection/
|   └─ rules.py
|   └─ correlator.py
| └─ sender/
|   └─ sender.py
└─ requirements.txt
```

3 WINDOWS LOG COLLECTOR

collectors/windows_logs.py

```
import win32evtlog

def collect_logs(limit=20):
    server = 'localhost'
    logtype = 'Security'

    hand = win32evtlog.OpenEventLog(server, logtype)
    flags = win32evtlog.EVENTLOG_BACKWARDS_READ |
win32evtlog.EVENTLOG_SEQUENTIAL_READ

    events = []

    records = win32evtlog.ReadEventLog(hand, flags, 0)

    for event in records[:limit]:
        events.append({
            "source": "windows_log",
            "event_id": event.EventID,
            "timestamp": str(event.TimeGenerated),
        })

    return events
```

4 PROCESS MONITOR (PID + PPID)

collectors/process_monitor.py

```
import psutil
```

```
def collect_processes():
    processes = []

    for proc in psutil.process_iter(['pid', 'ppid', 'name']):
        processes.append({
            "source": "process",
            "pid": proc.info['pid'],
            "ppid": proc.info['ppid'],
            "name": proc.info['name']
        })

    return processes
```

5 NETWORK MONITOR

collectors/network_monitor.py

```
import psutil

def collect_network():
    connections = []

    for conn in psutil.net_connections(kind='inet'):
        if conn.raddr:
            connections.append({
                "source": "network",
                "src_ip": conn.laddr.ip,
                "src_port": conn.laddr.port,
                "dst_ip": conn.raddr.ip,
                "dst_port": conn.raddr.port,
                "status": conn.status
            })

    return connections
```

6 DETECTION RULES

detection/rules.py

```

def detect(events):
    alerts = []

    for e in events:

        # Brute force
        if e.get("event_id") == 4625:
            alerts.append({
                "type": "BRUTE_FORCE",
                "severity": "HIGH"
            })

        # Suspicious process
        if e.get("name") == "powershell.exe":
            alerts.append({
                "type": "SUSPICIOUS_PROCESS",
                "severity": "MEDIUM"
            })

        # Network suspicious port
        if e.get("dst_port") == 4444:
            alerts.append({
                "type": "REVERSE_SHELL",
                "severity": "CRITICAL"
            })

    return alerts

```

7 CORRELATION ENGINE (REAL POWER)

detection/correlator.py

```

def correlate(processes, network):
    alerts = []

    for p in processes:
        for n in network:

            if n["src_ip"] == "127.0.0.1" and p["name"] == "powershell.exe":
                alerts.append({
                    "type": "LOCAL_ATTACK_CHAIN",
                    "details": f"{p['name']} communicating externally",

```

```
        "severity": "HIGH"  
    })
```

```
return alerts
```

8 SEND TO SIEM

sender/sender.py

```
import requests  
  
def send_alerts(alerts):  
    try:  
        requests.post("http://localhost:3001/logs", json=alerts)  
    except:  
        print("SIEM not reachable")
```

9 MAIN ENGINE

main.py

```
from collectors.windows_logs import collect_logs  
from collectors.process_monitor import collect_processes  
from collectors.network_monitor import collect_network  
  
from detection.rules import detect  
from detection.correlator import correlate  
from sender.sender import send_alerts  
  
def run():  
  
    logs = collect_logs()  
    processes = collect_processes()  
    network = collect_network()  
  
    combined = logs + processes + network  
  
    alerts = detect(combined)  
    correlation_alerts = correlate(processes, network)
```

```
all_alerts = alerts + correlation_alerts

print("ALERTS:", all_alerts)

if all_alerts:
    send_alerts(all_alerts)

if __name__ == "__main__":
    run()
```

REQUIREMENTS

requirements.txt

```
pywin32
psutil
requests
```

WHAT THIS AGENT DOES (REALISTIC)

✓ Collects:

- ✓ Windows Event Logs (auth, execution)
 - ✓ Running processes (PID, PPID)
 - ✓ Network connections
-

✓ Detects:

- ✓ Brute force (4625)
 - ✓ Suspicious processes (PowerShell)
 - ✓ Reverse shells (port 4444)
-

✓ Correlates:

Process + Network → Attack chain

WHAT THIS PROVES

- ✓ You built an endpoint monitoring agent
- ✓ You understand EDR concepts
- ✓ You correlate multiple telemetry sources
- ✓ You can detect real attack patterns